MY GITHUB PASSWORD - sahil@1909.
HTML:
HTML stands for Hyper Text Markup Language.
HTML is the code that is used to structure a web page and its content.
The component used to design the structure of websites are called HTML tags.

HTML Tags:
A container for some content or other HTML tags.
 This is a paragraph
content ``v vcxcxczqdaXQ
ELEMENT

BASIC HTML TAGS:
HTML Attirbutes:
Attributes are used to add more information to the tag

<html lang = "en">

HEADING TAG:
Used to display heading in HTML.
h1 (most important)
h2
h3
h4
h5
h6 (least important)

PARAGRAPH TAG:
Used to add paragraph in HTML.
This is a paragraph

ANCHOR TAG:
Used to add links to your page.
Youtube
Click Hello

IMAGE TAG:
Used to add image to your page.
<img src="https://m.media-amazon.com/images/I/317VTY6RngL._AC_UF894,1000_QL80_.jpg"alt = "Bat" height="200">

BR TAG:
Used to add new line (line breaks) to your page

BOLD, ITALIC & UNDERLINE TAGS
Used to highlight text in your page.
Hello everyone
Hello everyone
<u>Hello everyone</u>

BIG TAG & SMALL TAG:

Used to display big & small text on your page.
<big>This tag shows big font</big>
<small>This tag shows small font</small>

HR TAG:
Used to display a horizontal ruler, used to separate content.
<hr>

SUBSCRIPT & SUPERSCRIPT TAG:
Used for chemistry labels.
H₂O
3³= 9

PRE TAG:
Used to display text as it is (without ignorning spaces & next line)
<pre>
    Lorem   ipsum dolor sit amet consectetur adipisicing
elit. Illo accusantium eius ipsam laboriosam ipsa aperiam
asperiores, totam eaque similique ab iusto fugit facilis sint.
</pre>

PAGE LAYOUT TECHNIQUES:
Using semantic tags for layout
using right tags

<header>
<main>
<footer>

INSIDE MAIN TAG:
Section tag - for a section on you page.
<section>
Article tag - for an article on your page.
<article>
Aside tag - for contact aside main content(ads).
<aside>

REVISITING ANCHOR TAG:
  Google
                                    (for new tab)

<a href="https://www.google.co.in/" target="_main">
    <img src="https://encrypted-
tbn0.gstatic.com/images?q=tbn:ANd9GcS6WwgH7Nl5_AW9nDCnR2Ozb_AU3rkIbSJdAg&
s"
    alt="Google"
    height = "150"

DIV TAG: (non semantic tag)
Div is a container used for other HTML elements.
Block element(takes full width).

SPAN TAG:
Span is also a container used for other HTML elements.

Inline elements(takes width as per size).

LIST IN HTML:
Lists are used to represent real life list data.

*UNORDERED

```
<ul>
  <li>
    Apple
  <ul>
    <li>Healthy</li>
  </ul>
  </li>
  <li>
    Mango
  <ul>
    <li>Sweet</li>
  </ul>
  </li>
  <li>
    Banana
  <ul>
    <li>
      Healthy
    </li>
  </ul>
  </li>

</ul>
```

*ORDERED

```
</ul>

<ol>
  <li>
    Lichi
    <ul>
      Sweet
    </ul>
  </li>
  <li>
    Watermelon
    <ul>
      Hydreated
    </ul>
  </li>
</ol>
```

TABLES IN HTML:
Tables are used to represent real life table data.

<tr> used to display table row.
<td> used to display table data.
<th> used to display table header.

```
<table>
  <tr>
    <th>NAME</th>
    <th>CONTACT</th>
  </tr>
  <tr>
  <td>Sahil</td>
  <td>9920450858</td>
  </tr>
  <tr>
    <td>Akash</td>
    <td>8691938990</td>
  </tr>
</table>
```

CAPTION IN TABLE:

```
<caption> student data </caption
```

thead and tbody in tables:

```
<thead> to wrap the table head
<tbody> to wrap the table body.
```

FORM IN HTML:

Forms are used to collect data from the user.
Eg-signup/login/help requests/contact me.

ACTION IN FORM:

Action attribute is used to define what action needs to be performed when a form is submitted.

```
<form action= "/action.php">
```

FORM ELEMENT: INPUT

CLASS & ID:

CHECKBOX:

used to select all the options.

```
<label for="English">
    <input type="checkbox" class="English" name="subject"
id="102">English
        <br>
 </label>

<label for="math">
    <input type="checkbox" class="Math" name="subject" id="101">Math
 </label>
```

TEXTAREA TAG:

Used in html to write a feedback or text.

```
 <textarea name="feedback" id="101" placeholder="Please give your
valuable feedback here" rows="5">Feedback
```

```
 </textarea>

SELECT TAG:
select tag is used to select any given option.
 <select name="city">
            <option value="Delhi">Delhi</option>
            <option value="Mumbai">Mumbai</option>
            <option value="Uttar pradesh">Uttar pradesh</option>
            <option value="Pune">Pune</option>
            <option value="Bhusawal">Bhusawal</option>
            <option value="Thane">Thane</option>
            <option value="Malad">Malad</option>
        </select>

SUBMIT BUTTON:
Used to submit.
<input type="submit" name="submit">

IFRAME TAG:
Used to add website links in html.
<iframe width="560" height="315"
src="https://www.youtube.com/embed/mlIUKyZIUUU?si=4hqNwV7t6fo6_sNC"
title="YouTube video player" frameborder="0"
allow="accelerometer; autoplay; clipboard-write; encrypted-media;
gyroscope;
picture-in-picture; web-share" referrerpolicy="strict-origin-when-cross-
origin" allowfullscreen></iframe>

VIDEO TAG:
<video src="/video/.mp4"> My video </video>

Attributes:
controls
height
width
loop
autoplay

                                                        *****************

*CSS*
CASCADING STYING SHEET.
It is a language that is used to describe the style of a document.

Basic syntax:
                       Value
                       |
(selector)  h1{color:red;}
                 |
              Property

Including style:
*Inline
<h1 style="color:red;"> Apna College </h1>
```

*<Style> Tag
<style>
h1 {
color:red;}
</style>

*Before CSS we should link our HTML file with CSS file.
for ex-
<link rel="stylesheet" href="style.css">

EXTERNAL STYLESHEET:
Writing CSS in a separate document & and linking it with the HTML file.

COLOR PROPERTY:
Used to set the color of foreground.

color:red;
color:pink;
color:yellow;

BACKGROUND COLOR PROPERTY:
Used to set the color of background.

background-color:red;
background-color:blue;
background-color:pink;

COLOR SYSTEMS:
*RGB(red, green, blue).
color: rgb(255,0,0);   (red)
color: rgb(0.255.0);   (green)
color: rgb(0,0,255);   (blue)

```
body{
    background-color: rgb(255, 0, 0);
}
```

Color Systems
*Hex(Hexadecimal)

color:#ff0000; (red)
color:#00ff00: (green)
color:#0000ff; (blue)

```
body{
    background-color: #0000ff
}
```

TEXT PROPERTIES:
TEXT ALIGN:
text-align : Left/Right/Center.

TEXT DECORATION:

text-decoration : underline/overline/line-through.

FONT WEIGHT:
font-weight : normal/bold/bolder/lighter.
font weight : 100-900

FONT FAMILY/STYLE:
font-family : arial
font-family : arial,roboto.
We can write multiple fonts at a time.

UNITS IN CSS:
ABSOLUTE:
pixels (px)
96px = 1 inch
font-size : 2px

LINE HEIGHT PROPERTY:
line-height : 2px

line-height : normal

TEXT TRANSFORM:
text-transform : uppercase/lowercase/capitalize/none

BOX MODELS IN CSS:
Height
Width
Border
Padding
Margin

*Height-
By default, it sets the content area height of the element.

```
div {
height : 50px;
}
```

*Width-
By default, it sets the content area width of the element.

```
div {
width : 50px;
}
```

Border-
Used to set an element's border property.

border-width: 2px;
border-style: solid/dotted/dashed.
border-color: red;

BORDER SHORTHAND:

```
div {
border: 2px solid yellow;
}
```

BORDER-
Used to round the corners of an element's outer border edge.

```
border-radius : 50%
border-radius : 10px;
```

PADDING:

```
padding-left
padding-right
padding-top
padding-bottom
```

Padding shorthand:

```
padding:50px;
padding:1px 2px 3px 4px;  (clockwise)
      top right bottom left.
```

MARGIN:

```
margin-right
margin-left
margin-top
margin-bottom
```

MARGIN SHORTHAND:

```
margin: 25px 25px 25px 25px;  (clockwise)
```

DISPLAY PROPERTIES:
display: inline/block/inline-block/none.
Block elements are - div, H1 etc.
inline elements are - span, input, button, anchor tags etc.

*inline - Takes only place requiered by the element.(no padding/no margin).
*block - Takes full space avalaible in width.
*inline-block-Similiar to inline but we can set margin and padding.
*none - to remove the element from the document from the flow.

VISIBILITY :
Note-when visibility is set to none,space for the element is reserved. But for display set to none, no space is reserved or blocked to the element.
visibilty: hidden:

ALPHA CHANNEL:
opacity(0 or 1)
*RGBA

```
color:rgba(255,0,0,0.5);  (light red)
color:rgba(255,0,0,1);  (red)
```

UNITS IN CSS;
Relative

%
em
rem

Percentage(%)
It is often used to define a size as relative to an element's parent object.

width: 33%;
margin-left:50px;

EM:
unit-em
Relative to- Font size of the parent, in the case of typographical properties like font-size, and font size of the element itself, in the case of other properties like width.

REM *(Root em):
Unit - rem
Relative to- Font size of the element.

POSITION:
The Position CSS property sets how an element is positioned in a document.
Position: static/ relative/ absolute/ fixed.

POSITION:
*Static- default position(the top,right,bottom,left and z-index properties have no effect).

*relative- element is relative to itself.(the top,right,bottom,left and z-index will work)

*absolute- position relative to its closest positioned ancestor.(removed from

*fixed- positioned relative to browser.(removed from flow).

*sticky- positioned based on user's scroll position.

*Z-INDEX:
It decides the stack level of elements
Overlapping elements with a largr z-index cover thods wth a smaller one.

z-index : auto (0)
z-index : 1/2/...
z-index : -1/-2/...

BACKGROUND IMAGE:
backgrounf-image : url("phto.jpg")

BACKGROUNF SIZE PROPERTY:
background-size : cover/contain/auto.

FLEX BOX:
Flexible box layout.
It is one-dimensional layout methoud for arranging items in rows or colums.

FLEXBOX DIRECTION:
It sets how flex items are placed in the flex container, along which axis and direction. (main axis, cross axis)

```
justify-content: center;
align-items : center;
```

MEDIA QUERIES:
Help to create a responsive website.

for ex-

```
@media (width:700px) {
    div {
        background-color: aqua;
    }
}
@media (min-width : 300px) and (max-width : 500px) {
    div {
        background-color: yellow;
    }
}
```

TRANSISION:
Transition enable you to define the transition between two states of an element.

transition property : property you want to transition(font-size, width etc)
transition-duration : 2s/4mns..
transition-timming-function : ease-in/ease-out/ linear/steps..
transion-delay : 2s/2mns..

CSS TRANSFORM:
Used to apply 2D&3D transformations to an end to an element.

*rotate

```
transform: rotate(45deg):
rotate: 45deg;
rotateX: 45deg;
rotateY: 45deg;
rotateZ: 45deg;
```

*scale

```
transform: scale(2);
transform: scale(0.5);
transform: scale(1,2);
```

```
transform: scaleX(0.5);
transform: scaleY(0.5);

ANIMATION:
To animate CSS elements.

ANIMATION PROPERTIES:
animation-name
animation-duration
animation-timing-function
animation-delay
animation-iteration-count
animation-direction

for ex-
animation-name: sahilanimate;
animation-timing-function:ease-in;
animation-duration: 1s;
animation-delay: 1s;
animation-iteration-count: 5;
animation-direction: normal;


@keyframes sahilanimate {
    from {background-color: yellow;}
    to {background-color: red;}
}




****************************************
*JAVASCRIPT*
*Javascript is a programming language. We use it to give instruction to
the computer/laptop.
JavaScript is a dynamic language.

"=" is an assignment operator.

Chapter 1:
Varaibles and Data types.
Varaibles are containers for data.

*LET, VAR & CONST:

var- variable can be re-declared & updated. A global scope variable.

let- variable cannot be re-declared but can be updated.A block scope
variable.

const- variable cannot be re-declared or updated. A block scope variable.

*DATA TYPES IN JS:
```

This all are primitive data types.
Number, String, Boolen, Undefined, Null, BigInt, Symbol.

*OBJECT:
Object is a collection of values.
for ex-

```
const student = {
    name : "Sahil",
    age : 22,
    isPass : true,
    education : "BSCit"
};
```

CHAPTER 2:
*OPERATORS & CONDITIONAL STATMENTS:

*COMMENTS IN JS:
Part of code which are not executed.

for ex-

```
//this is a comment.
```

*MULTI-LINE COMMENT:
for ex-

```
/* This is a multi-line
comment */
```

*OPERATORS:
Used to perform some operation on data.
Arthimetic operators are:
+,-,*,/

1)Modulus (%)
2)Exponentional (**)
(unary operators)
3)Increment (++)
4)Decrement (--)

*ASSIGNMENT OPERATORS:
=, +=, -=, *=, **=, %= .

*COMPARISON OPERATORS:
1)equal to ==
2)not equal to !=
3)equal to & type ===
4)not equal to & type !==

>,>=,<=,<

*LOGICAL OPERATORS:

1)Logical AND &&
2)Logical NOT !

3)Logical OR ||

*CONDITIONAL OPERATORS:
To implement some condition in the code.

*IF STATEMENT:;
for ex-

```
let age = 20;

if(age>=18) {
    console.log("You can drive");
}

if(age<=18) {
    console.log("You cannot drive");
}
```

*IF-ELSE STATEMENT:
for ex-

```
let mode = "white"
let color;

if (mode === "black") {
    color = "Black";

} else {
   color = "White";
}

console.log(color)
```

*ELSE IF STATEMENT:
for ex-

```
let mode = "red";

if (mode === "black") {
    console.log("The color is black");

} else if (mode === "blue") {
    console.log("The color is Blue")

} else {
    console.log("The color is different")
}
```

*TERNARY OPERATORS:
condition ? true output : false output;
for ex-

```
let age = 2;

let result = age >= 18 ? "Is adult" : "Not adult";
console.log(result)
```

CHAPTER 3:
LOOPS & STRINGS:

*LOOPS IN JS:
Loops are used to exceute a piece if code again and again.

*FOR LOOP:
for ex-

```
for (let i = 1; i<=5; i++) {
    console.log("Sahil")
}
```

*WHILE LOOP:
for ex-

```
i = 1
while (i<=5) {
    console.log("i = ", i)
    i+= 1
}
```

*DO WHILE LOOP:
The code exceutes atleast one time.
for ex-

```
i = 20;

do {
    console.log("JavaScript");
    i++
} while (i <= 10);
```

*FOR OF LOOP:
Used for String and Arrays.
There's no need for initialation, updation and stopping condition.It is a iterator.
for ex-

```
let str = "sahilsoni"

for (let i of str) {
    console.log("i = ", i)
}
```

*FOR IN LOOP:
Used for Object and Array.
for ex-

```
let student = {
    name : "Sahil Soni",
    age : 22,
    cgpa : 8.6,
```

```
    isPass : true
}

for (key in student) {
    console.log("key=", key, "Value=", student[key]);
}
```

*STRINGS IN JS:
Strings is a sequence of characters used to represent text.

*Create string:

```
let str = "Hey this is a string"
```

*String lenght:

```
let str = "Sahil"
console.log(str.length)
```

*String Indics:

```
let str = "Sahil"
console.log(str[0])
```

*TEMPLATE LITERALS IN STRING:
A way to have embedded expressions in strings.
for ex-

```
`This is a template literal`
```

*String interpolation:
To create strings bt doing subsitution of placeholders.
for ex-

```
let obj = {
    item : "pen",
    price : 20,
};
let output = `The cost of ${obj.item} is ${obj.price}`;
console.log(output)
```

*STRING METHOUDS IN JS:
These are inbuilt functions to manipulate a string.

```
1)str.touppercase();
2)str.tolowercase();
3)str.trim();  //removes white space.
4)str.slice(start,end); //returns part of string
5)str1.concat(str2);
6)str.replace(searchval,newvalue);
7)str.charAt(idx);
```

*ARRAYS IN JS:
It is a collection of items.
Strings in arrays are muttable.

*ARRAY METHOUD:

1)push()...add to end.
2)pop().. to delete from end.
3)toString..converts an array into string.
4)concat()...joins multiple array and returns result.
5)unshift()...add to start.
6)shift()...delete from start.
7)slice()...returns a piece of array. (startidx,index)
7)splice()...change original array.(add,remove,replace)

*FUNCTIONS IN JS:
Function is a block of code that performs a specific task, can be invoked whenever needed.
for ex-

```
function calSum(a,b) {
    console.log(a+b);
}

calSum(2,3);
```

*ARROW FUNCTION:
Compact way of writing a function.
for ex-

```
const arrowSum = (a,b) => {
    console.log(a+b);
}

console.log(arrowSum(2,5));
```

*FOREACH LOOP: (can only be use with array)
for ex-

```
arr = [1,2,3,4,5];

arr.forEach((val)=> {
    console.log(val);
});
```

FUNCTIONS IN JS:
Block of code that performs a specific task, can be invoked whenever needed.

ARROW FUNCTIONS:
It is an compact way of writing function.
for ex-

```
const sum = (a,b) => {
    return a+b;
}

let val = sum(4,5);
console.log("Sum of two numbers is", val);
```

FOR EACH LOOP:

Can only be use in an array.
for ex-

```
arr = ["sahil", "pratik", "fapale", "piyush"];

arr.forEach((val) => {
    console.log(val);
});
```

SOME MORE ARRAY METHOUDS:
1)MAP:
Creates a new array with the results of some operation. The value its callback returns are used to form new array.
for ex-

```
arr = [24,67,98,76];

let newArr = arr.map((val) => {
    console.log(val * 2);
})
```

2)FILTER:
Creates new array of elements that gives true for a condition/filter.
for ex-

```
arr = [1,2,3,4,5,6,7,8];

let evenArr = arr.filter((val) => {
    return val % 2 === 0;
})


console.log(evenArr);
```

3)REDUCE:
Performs some operations & reduces the array to a single value. It returns that single value.
for ex-

```
arr = [1,2,3,4];

let evenArr = arr.reduce((res,curr) => {
    return res + curr;
})

console.log(evenArr);  //output - 10
```

program to find the largest number in an array:
ex-

```
let arr = [12,45,76,88,98];

let largestNum = arr.reduce((pev,curr) => {
    return pev > curr ? pev : curr;
```

```
})

console.log(largestNum);   //output 98
```

WINDOW OBJECT:
The window object represents an open window in a brower. It is brower's object(not javascript) & automatically created by brower.
It is global object with lots of properties & methods.

WHAT IS DOM ?
When a web page is loaded, the brower creates a DOCUMENT OBJECT MODEL(DOM) of the page.
It is a tree like structure.

DOM MANIPULATION:
Selecting with id-

```
document.getElementByid("myid")
```

Selecting with class-

```
document.getElementsByclassName("myClass")
```

Selecting with tag-

```
document.getElementsbyTagName("p")
```

DOM MANIPULATION:
PROPERTIES:
1)tagName - returns tag for element name.
2)innerText - returns the text contentof the element and all its children.
3)innerHTML - returns plain text or HTML contents in the element.
4)textContent - returns text content even for hidden elements.

ATTRIBUTES:

```
getAttribute(attr)  //to get the attribute value.
getAttribute(attr,value)  //to set the attribute val th
```

STYLE:

```
node.style  //to add styling.
```

INSERT ELEMENTS:
1)node.append(el) //adds at the end of node(inside)
2)node.prepend(el) //adds at the start of node(inside)
3)node.before(el) //adds before the node(outside)
4)node.after(el) ..adds after the node(outside)

DELETE ELEMENT:

```
node.remove() //removes the node.
```

*EVENTS IN JS:
The change in the state of an object is known as Event.
Events are fired to notify code of "interesting changes" that may affect code execution.

*DESTRUCING:

Destructuring in JavaScript is a syntax that allows you to unpack values from arrays or properties from objects into distinct variables.
It simplifies the process of extracting multiple properties or elements, making the code more readable and concise.

for ex-

```
const person = {
  name: "Alice",
  age: 25,
  city: "New York"
};

// Destructuring the object into variables
const { name, age, city } = person;

console.log(name); // Alice
console.log(age);  // 25
console.log(city); // New York
```

ex2-

```
const arr = [1, 2, 3, 4];

// Destructuring the array into variables
const [a, b, c] = arr;

console.log(a); // 1
console.log(b); // 2
console.log(c); // 3
```

HOISTING:
Hoisting in JavaScript is a behavior where variable and function declarations are moved (or "hoisted") to the top of their containing scope during the compilation phase, before the code execution begins. This means that you can reference variables and functions even before they are declared in the code. However, only the declarations are hoisted, not the initializations.

for ex-

```
console.log(a); // undefined (not ReferenceError)
var a = 5;
console.log(a); // 5
```

CLASESS AND OBJECTS:

PROTOTYPES IN JS:
A javascript is an entity having state and behaviour (properties and methoud).
JS objects have a special property called prototype.
We can set prototype using __proto__
for ex-

```
const employee =  {
```

```
    fullName : "Sahil Soni",
    calName() {
        console.log("My name is", this.fullName)
    }

};

// console.log(employee.fullName);

const student = {
    stName : "Pratik",
    age : 24,
    calAge() {
        console.log("My age is", this.age);
    }
}

employee.__proto__ = student;

CLASSES IN JS:
Class is a program-code template for creating objects.
those objects will have some state(variables) & some behaviour(functions)
inside it.
for ex-

class car {
    start() {
        console.log("Car started!");
    }

    stop() {
        console.log("Car stoped!");
    }

    model(brand) {
       this.brand = brand
    }
};

let fortuner = new car();

let lexus = new car();

fortuner.model("Fortuner");
lexus.model("lexus");

CONSTRUCTOR IN JS:
Constructor() methoud is : automatically invoked by new, initializes
object.
for ex-

class car {
    constructor(brand, mileage) {
        console.log("Creating new objects");    #constructor
```

```
        this.brand = brand;
        this.mileage = mileage;
    }
    start() {
        console.log("Car started!");
    }

    stop() {
        console.log("Car stoped!");
    }

};

let fortuner = new car("fortuner", 10);
let lexus = new car("lexus", 12);

INHERITANCE IN JS:
Inheritance is passing down properties & methods from parent class to
child class.
for ex-

class person {
    eat() {
        console.log("Time to eat");

    }
    sleep() {
        console.log("Time to sleep");
    }
}

class engineer extends person{
    work() {
        console.log("Work hard, acheive success");
    }
}

let sahilobj = new engineer();

SUPER KEYWORD:
The super keyword is used to call the constructor of its parent class to
access the parent's properties and methouds.
for ex-

class person {
    constructor(name, age) {
        this.name = name;
        this.age = age;
    }
    eat() {
        console.log("Time to eat");

    }
    sleep() {
```

```
        console.log("Time to sleep");
    }
}

class engineer extends person{
    constructor(name, age) {
        super(name, age)
    }
    work() {
        super.eat()
        super.sleep()
        console.log("Work hard, acheive success");
    }
}

let sahilobj = new engineer("Sahil", 22);
```

ERROR HANDLING:
try-catch
for ex-

```
let a = 2;
let b = 5;

console.log(a+b);
console.log(a+b);

try {
    console.log(a+c);
} catch(err) {
    console.log("error");
}
console.log(a+b);
console.log(a+b);
```

SYNC IN JS:
Synchronus means the code runs in a particular sequence of instructions given in the program.Each instruction waits for the previous instruction waits for the previous instruction to complete its execution.

ASYNCHRONUS:
Due to synchronus programming, sometimes imp instructions get blocked due to some previous instruction, which causes a delay in the UI.
Asycnchronus code exceution allows to exceaute nect instructions immediately and doesn't block the flow.

SETTIMEOUT FUNCTION:
It is used to set time taken to show the output. Can only be use in asynchronus programs.
for ex-

```
function hello() {
    console.log("Hello!");
}
```

```
setTimeout(hello, 3000);
```

CALLBACK HELL:
Callback hell : Nested callbacks stacked below one anither forming a pyramid structure.(pyramid of doom).
This style of programming becomes difficult & manage.
for ex-

```
function getData(dataId, getNextData) {
    setTimeout(() => {
        console.log("Data", dataId);
        if(getNextData) {
            getNextData();
        }
    },2000);
}

getData(1, () => {
    console.log("Getting data ..2")
    getData(2, () => {
        console.log("Getting data ..3")
        getData(3, ()=> {
            console.log("Getting data ..4")
           getData(4, () => {
            console.log("Data receivd");
           })
        })
    })
})
```

PROMISE:
Promise is for "eventual" completion of task. It is an object in JS. It is a solution to callback hell.
for ex-

```
let promise = new Promise((resolve, reject) => {
    console.log("Hi i am a promise");
    resolve("Success")
})
```

for ex2-

```
function getData(getData, getNextData) {
    return new Promise((resolve, reject) => {
        setTimeout(() => {
            console.log('Data', getData);
            resolve("Success");
            if(getNextData) {
                getNextData();
            }
        }, 3000);
    });
}
```

PROMISES:
then() & catch()
for ex-

```
const getPromise = () => {
    return new Promise((resolve, reject) => {
        console.log("I am a promise")
        reject("Network error")
    })
}

let promise = getPromise();
promise.then((res) => {
    console.log("Promise fullfilled", res);
})

promise.catch((err) => {
    console.log("Some error occuired", err);
})
```

ASYNC-AWAIT:
Async function always return a promise.
await pauses the exceution of its surrounding async function until the promise is settled.
for ex-

```
function getWeather() {
    return new Promise((resolve, reject) => {
       setTimeout(() => {
        console.log("Weather data");
        resolve("success/200");
       },3000);
    });
}


async function getData() {
    await getWeather(1);
    await getWeather(2);
    await getWeather(3);

}
```

Fetch API'S: (Application Programming Interface)
The fetch API provides an interface for fetching(sending/ recieving) resources.
It uses Request and Respone objects.
The fetch() method is used to fetch resources(data).

```
let promise = fetch(url.[optional])
```

UNDERSTANDING TERMS:
AJAX is Asynchronus JS & XML.
JSON is JavaScript Object Notation

JSON method : returns a secound promise that resolves with the result of parsing the response body text as JSON. (input is JSON, output is JS object

```
/* body {
    background-color: rgb(0, 0, 33);
    color: white;
   }

   nav {
    display: flex;
    justify-content: space-evenly;
    background-color: rgb(18, 18, 62);
    border: 1px solid;
   }

   .container {
    height: 50px;
    width: 400px;
    display: flex;
    align-items: center;
    }

    a {
        text-decoration: none;
        color: white;
        margin-right: 100px;
    }

    a:hover {
        border: 1.5px solid white;

    }

    .name {
        position: absolute;
        left: 150px;
        font-size: 1.5rem;
    }

    .first {
        display: flex;
        justify-content: center;
        margin: 23px 0;
    }

    .left {
        width: 34%;
        font-size: 2.5rem;
        margin-top: 150px;
        position: absolute;
        left: 100px;
    }
```

```
.left div {
    font-size: 2.5rem;
}

.right {
    width: 35%;
    position: absolute;
    right: 100px;
}

.purple {
    color: rgb(170, 107, 228);
}

span {
    color: rgb(170, 107, 228);
} */
```